SenEvent

apps/web · **apps/mobile** · packages/shared

DAWM · BLOC DÉPLOIEMENT 2/2 · M1 RETEL — DMI/FST/UCAD

Lab 12 — Build mobile avec EAS : l'APK Android

Rapport de réalisation : l'application mobile quitte Expo Go. EAS Build l'a compilée sur les serveurs d'Expo en un APK installable, qui embarque le même package partagé que le site Vercel et parle à la même base. La session de l'utilisateur survit désormais à la fermeture de l'application, et l'APK sait recevoir des corrections JavaScript sans être reconstruit.

tag : v2.0 | APK 1.0.0 · versionCode 1 | sn.ucad.senevent.mbow | Expo SDK 57.0.0 | AsyncStorage | EAS Update · channel preview

01 D'Expo Go à une application compilée

Expo Go exécute le JavaScript que lui envoie le serveur de développement d'un ordinateur : l'ordinateur éteint, l'application disparaît. Un APK contient tout — le moteur React Native, les modules natifs et le JavaScript assemblé une fois pour toutes — et porte son propre nom et son propre identifiant. EAS Build fait la compilation Android (Gradle) sur les serveurs d'Expo : aucune installation d'Android Studio n'a été nécessaire.

	EXPO GO (LABS 8–11)	APK (LAB 12)
Le JavaScript vient de	le serveur Metro de l'ordinateur	l'APK lui-même (bundle Hermes)
Les cles Supabase viennent de	apps/mobile/.env, lu par Metro	l'environnement EAS preview, au build
La session est gardee	en memoire (perdue a la fermeture)	dans AsyncStorage
Identite	celle d'Expo Go	SenEvent · sn.ucad.senevent.mbow

02 Préparer le projet : expo-doctor et app.json

`expo-doctor` signalait deux paquets en retard d'un correctif sur le SDK 57 (`expo` 57.0.30, `expo-router` 57.0.28), corrigés avec `npx expo install --fix`. Le seul contrôle restant porte sur la version de Xcode de la machine : il concerne les builds iOS locaux, pas le build Android d'EAS. L'identité de l'application :

CHAMP	VALEUR	RÔLE
name	`SenEvent`	`le nom sous l'icone`
version	`1.0.0`	`la version visible ; elle sert aussi de runtimeVersion a EAS Update`
android.package	`sn.ucad.senevent.mbow`	`identifiant definitif de l'application sur Android`
owner · projectId	`ahmadmbow2004 · a38d166b-643b-4108-bd7c-3a111d4bc7a6`	`ajoutes par eas init : toute l'equipe vise le meme projet EAS`

Le compte Expo est `ahmadmbow2004`. La connexion du CLI s'est faite par la méthode « appareil » (`eas login --device`) : le terminal de l'outil n'étant pas interactif, l'utilisateur a validé un code dans son navigateur, sans qu'aucun mot de passe ne transite par l'outil.

03 Les profils de build

APPS/MOBILE/EAS.JSON

```
{
  "cli": {
    "version": ">= 16.0.0",
    "appVersionSource": "remote"
  },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal",
      "channel": "development"
    },
    "preview": {
      "distribution": "internal",
      "environment": "preview",
      "android": {
        "buildType": "apk"
      },
      "channel": "preview"
    },
    "production": {
      "environment": "production",
      "autoIncrement": true,
      "channel": "production"
    }
  },
  "submit": {
    "production": {}
  }
}
```

Le profil `preview` produit un APK (`buildType: apk`) au lieu de l'AAB du Play Store, lit les variables de l'environnement EAS `preview`, et publie ses mises à jour sur le channel `preview` (bonus). `appVersionSource: remote` laisse EAS tenir le compteur de versions : le premier build a initialisé le versionCode à 1.

04 Les variables d'environnement EAS

Le piège du Lab 11 se rejoue : `apps/mobile/.env` est ignoré par Git, donc absent de ce qu'EAS envoie. Avant la déclaration, `eas config` l'annonçait déjà : « No environment variables … found for the "preview" environment ». Les deux variables ont été envoyées depuis le `.env` par `eas env:push preview --path .env`, lancée par l'utilisateur : leurs valeurs ne sont jamais passées par l'outil.

VARIABLE	VISIBILITÉ	ENVIRONNEMENT
EXPO_PUBLIC_SUPABASE_ANON_KEY	PUBLIC	preview
EXPO_PUBLIC_SUPABASE_URL	PUBLIC	preview

Le journal du build l'a confirmé au lancement : *Environment variables … loaded from the "preview" environment on EAS: EXPO_PUBLIC_SUPABASE_ANON_KEY, EXPO_PUBLIC_SUPABASE_URL*. Et l'APK final les contient bien (section 08) — comme `VITE_` sur le web, une variable `EXPO_PUBLIC_` est lisible par quiconque extrait l'APK : sans danger pour la clé publique protégée par la RLS, et c'est pourquoi `service_role` n'y figure pas.

05 Une session qui survit à la fermeture

React Native n'a pas de `localStorage` : sans stockage fourni, supabase-js garde la session en mémoire, et l'utilisateur est déconnecté à chaque fermeture — la limite relevée au Lab 10. Le package partagé accepte maintenant un troisième paramètre, optionnel :

PACKAGES/SHARED/SRC/SUPABASECLIENT.TS

```
export function initSupabase(
  url: string,
  anonKey: string,
  options?: SupabaseClientOptions<"public">
): SupabaseClient {
  if (!client) {
    client = createClient(url, anonKey, options);
  }
  return client;
}
```

Le web n'a pas changé : `initSupabase( import.meta.env.VITE_SUPABASE_URL, import.meta.env.VITE_SUPABASE_ANON_KEY );`. Seul le mobile passe un stockage :

APPS/MOBILE/APP/_LAYOUT.JS

```
initSupabase(
  process.env.EXPO_PUBLIC_SUPABASE_URL,
  process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY,
  {
    auth: {
      storage: AsyncStorage,
      persistSession: true,
      autoRefreshToken: true,
      // Pas de session a lire dans une URL de page, contrairement au web.
      detectSessionInUrl: false,
    },
  }
);
```

La preuve, sans téléphone. Node n'a pas de `localStorage` non plus. `apps/mobile/test-session-lab12.mjs` regroupe le vrai package partagé et le fait tourner dans deux processus séparés : le premier se connecte, le second — tout neuf, comme une application rouverte — relit la session. Le stockage de test reprend les trois méthodes asynchrones d'AsyncStorage, écrites sur disque.

INITIALISATION	CE QUI EST STOCKÉ	APRÈS LE « REDÉMARRAGE »
initSupabase(url, key) — Lab 10	`rien`	`session perdue (null)`
initSupabase(url, key, { auth: { storage } }) — Lab 12	`sb-siszafptxskyfdedpqsl-auth-token`	`session retrouvee (ahmadoussj+…@gmail.com)`

Côté web, la connexion survit toujours au rechargement (clé `sb-…-auth-token` dans le `localStorage`), et la préversion Vercel de la PR #7 — qui touchait `packages/shared` — a construit le site sans erreur. Le stockage de test, qui contient un jeton, est supprimé à la fin du test.

06 Ce qui part sur les serveurs d'Expo

EAS envoie tout le dépôt, depuis la racine, à l'exception de ce que `.gitignore` exclut. Avant le build, l'archive a été produite localement par `eas build:inspect --stage archive` et fouillée. Un dossier y figurait à tort : `supabase/fixtures/`, jamais versionné mais pas ignoré non plus, qui contient le mot de passe des comptes de test. Il a été ajouté au `.gitignore` (PR #9) avant le build.

DANS L'ARCHIVE ENVOYÉE	RÉSULTAT
apps/mobile/.env, apps/web/.env.local	absents
supabase/fixtures/ (apres la PR #9)	dossier vide, aucun fichier
mot de passe des comptes de test	0 fichier
cle publique Supabase	0 fichier (elle vient des variables EAS)
service_role	0 fichier de code

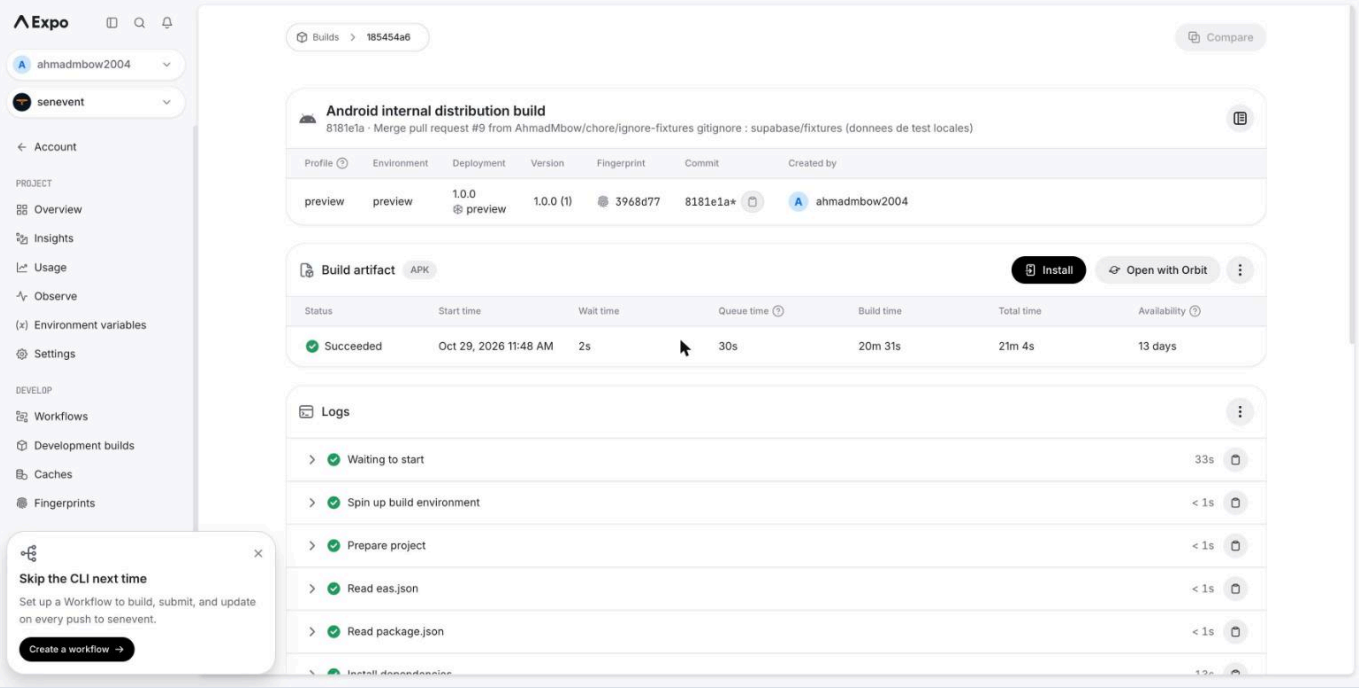
07 Le build

Lancé une seule fois, depuis `main` à jour (après les PR #7, #8 et #9), pour préserver le quota gratuit. Au premier build, EAS a généré le keystore Android — la clé qui signe l'application — et le conserve pour les suivants.

	VALEUR
Statut	FINISHED
Profil · distribution	preview · INTERNAL
Version · versionCode	1.0.0 · 1
SDK · runtimeVersion · channel	57.0.0 · 1.0.0 · preview
Duree totale (file d'attente comprise)	21.1 min
Commit construit	8181e1a
Temps de compilation · file d'attente	20.5 min · 31 s
Lien de l'APK valable jusqu'au	2026-11-12
Page du build	https://expo.dev/accounts/ahmadmbow2004/projects/senevent/builds/185454a6-f134-4979-b50c-fc5318437371

FIGURE 01 `expo.dev`

La page du build



Le bouton Install et le QR code pointent vers l'APK : c'est le lien donne dans le README.

08 Ce que contient l'APK

L'APK (105.8 Mo) a été téléchargé depuis EAS et ouvert, sans être exécuté : c'est l'application que recevra le téléphone.

CONTRÔLE	OÙ	RÉSULTAT
Bundle JavaScript embarque	`assets/index.android.bundle`	`present (Hermes)`
Identifiant Android	`AndroidManifest.xml`	`sn.ucad.senevent.mbow`
URL Supabase injectee	`bundle`	`presente`
Cle publique injectee	`bundle`	`presente`
service_role	`bundle`	`absente`
URL de mise a jour (bonus)	`AndroidManifest.xml`	`u.expo.dev presente`

La troisième ligne écarte le piège de l'étape 4 : sans les variables EAS, l'application se serait arrêtée au démarrage sur « supabaseUrl is required ».

09 La recette finale

La recette se fait sur un téléphone Android, ordinateurs éteints, en 4G : elle revient à l'équipe. La dernière colonne indique ce qui a été vérifié en amont, sur le même code ou sur l'APK lui-même.

#	ACTION	ATTENDU	VÉRIFIÉ EN AMONT	ÉTAT
1	Ouvrir l'application SenEvent	liste des evenements chargee	`l'APK contient l'URL Supabase et la cle : pas de « supabaseUrl is required »`	**à faire**
2	Toucher un evenement	la fiche de detail s'affiche	`parcours du Lab 9, meme code`	**à faire**
3	Onglet Profil, se connecter	email affiche, bouton de deconnexion	`parcours du Lab 10, meme code`	**à faire**
4	Fermer completement l'application, la rouvrir	toujours connecte	`test des deux processus : session retrouvee avec AsyncStorage`	**à faire**
5	Creer un evenement avec le bouton +	il apparait dans la liste	`parcours du Lab 10`	**à faire**
6	Ouvrir le site Vercel sur un autre appareil	l'evenement de l'etape 5 y est	`demo croisee du Lab 10 et recette du Lab 11`	**à faire**
7	Creer un evenement sur le site	il apparait dans l'app au retour sur la liste	`useFocusEffect, Lab 10`	**à faire**
8	S'inscrire a un evenement dans l'app	bouton « Se desinscrire »	`parcours du Lab 10`	**à faire**
9	Se deconnecter, toucher un evenement	invitation a se connecter	`parcours du Lab 10`	**à faire**
10	Installer l'APK sur un autre telephone	meme comportement	`meme APK, lien public`	**à faire**

10 Bonus : mettre à jour sans recompiler

`expo-updates` a été installé *avant* l'unique build : l'APK interroge ainsi `u.expo.dev` à l'ouverture et télécharge le JavaScript publié sur son channel, pour la même `runtimeVersion` (1.0.0). Une correction de texte — le sous-titre de l'écran Profil, qui donne désormais l'adresse du site — a été fusionnée (PR #11) puis publiée :

```
eas update --channel preview --environment preview --platform android \
  --message "corrige le sous-titre du profil : adresse du site"
```

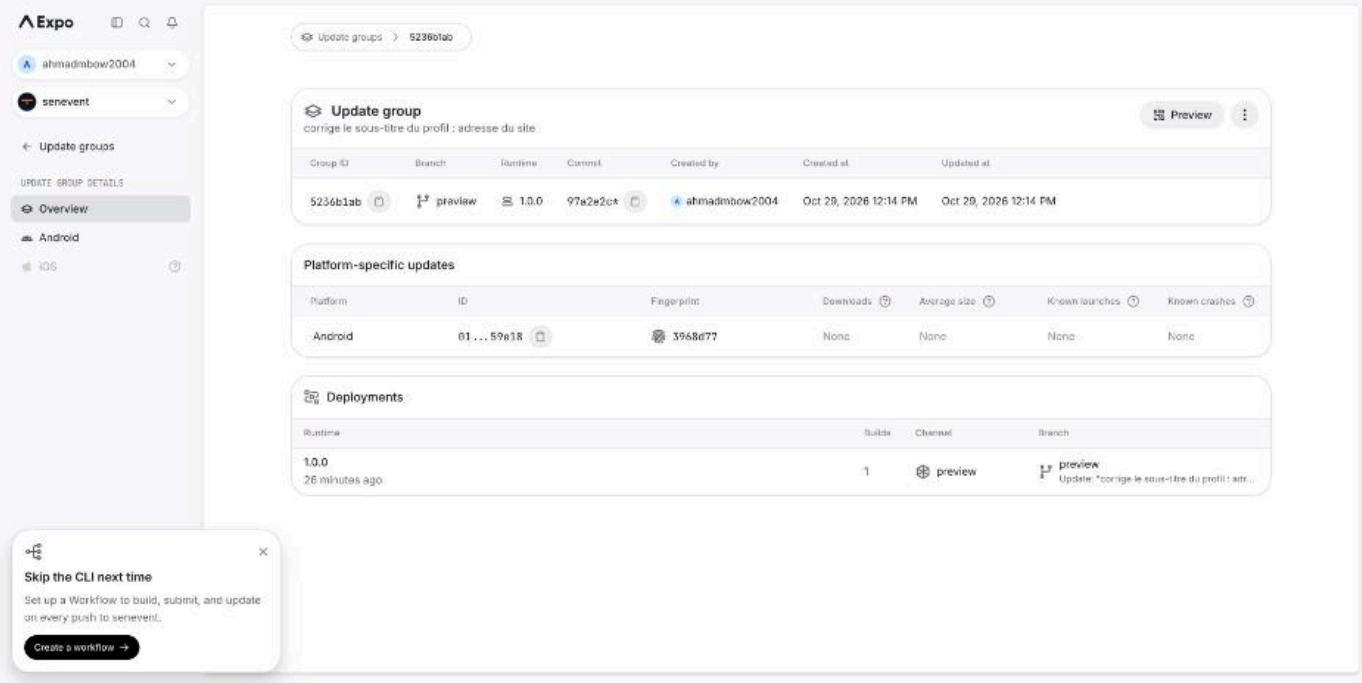
`--platform android` s'ajoute à la commande du sujet : `eas update:configure` n'a défini la `runtimeVersion` que pour Android, la seule plateforme construite ici.

MISE À JOUR PUBLIÉE	VALEUR
Message	`corrige le sous-titre du profil : adresse du site`
Branche · runtimeVersion	`preview · 1.0.0`
Plateformes	`android`
Compatible avec l'APK	`oui (meme runtimeVersion)`

Il faut fermer et rouvrir l'application deux fois : la première ouverture télécharge la mise à jour, la seconde l'applique. Limite à connaître : seul le JavaScript se met à jour ainsi ; un nouveau paquet natif, comme AsyncStorage, exige un nouveau build.

FIGURE 02 `expo.dev`

La mise à jour sur le channel preview



Publiee sans nouveau build ; l'APK installe la recupere a la prochaine ouverture.

11 Ce que la validation a réellement donné

Les vérifications de l'enseignant (section 13.2), l'état d'EAS et le contenu de l'APK, relus au moment de générer ce rapport :

CONTRÔLE	COMMANDE / MÉTHODE	RÉSULTAT	OK
Profil preview : APK, environnement preview	`grep -A4 '"preview"' apps/mobile/eas.json`	`"preview": { "distribution": "internal", "environment": "preview", "android": { "buildType": "apk" }, "channel": "preview" }, "production": { "environment": "production", "autoIncrement": true,`	✓
Identifiant Android et projet EAS	`grep -n '"package"|projectId' app.json`	`14: "package": "sn.ucad.senevent.mbow", 42: "projectId": "a38d166b-643b-4108-bd7c-3a111d4bc7a6"`	✓
Le mobile passe AsyncStorage	`grep -n AsyncStorage apps/mobile/app/_layout.js`	`4, 14, 20 (lignes)`	✓
initSupabase accepte des options	`grep -n options supabaseClient.ts`	`3 lignes`	✓
Aucun .env versionne	`git ls-files | grep -c "\.env"`	`0`	✓
Le README donne l'APK	`grep -n APK README.md`	`4 lignes`	✓
Aucun appel Supabase direct	`git grep "supabase.from|createClient" -- apps/mobile`	`0 occurrence`	✓
Variables EAS de l'environnement preview	`eas env:list --environment preview`	`EXPO_PUBLIC_SUPABASE_ANON_KEY (PUBLIC) · EXPO_PUBLIC_SUPABASE_URL (PUBLIC)`	✓
Le build preview a reussi	`eas build:view 185454a6…`	`FINISHED · 1.0.0 (1) · INTERNAL`	✓
L'APK porte le bon paquet	`AndroidManifest.xml de l'APK`	`sn.ucad.senevent.mbow`	✓
Les variables sont dans l'APK	`bundle Hermes de l'APK`	`URL Supabase : oui · cle publique : oui`	✓
service_role absent de l'APK	`bundle Hermes de l'APK`	`absent`	✓
La session survit au redemarrage	`test-session-lab12.mjs`	`sans option : perdue · AsyncStorage : retrouvee`	✓
Bonus : l'APK sait se mettre a jour	`manifeste + eas update:list`	`URL u.expo.dev : oui · 1 mise(s) a jour sur preview`	✓

Le lab compte 7 commits, passés par 5 Pull Requests (#7, #8, #9, #10, #11), chacune construite par GitHub Actions et prévisualisée par Vercel avant sa fusion.

12 Écarts et points à savoir

LE SUJET	ICI	POURQUOI
sn.ucad.senevent.equipe1	`sn.ucad.senevent.mbow`	`coherent avec le projet Vercel senevent-mbow`
eas login	`eas login --device`	`le terminal de l'outil n'est pas interactif : validation d'un code dans le navigateur`
eas env:set (x2)	`eas env:push preview --path .env`	`les valeurs vont du .env a EAS sans etre recopiees`
deux PR (A puis B)	`une PR pour A et B, puis une pour le bonus`	`meme ordre de fusion, avant le build`
—	`supabase/fixtures/ ajoute au .gitignore`	`sinon EAS envoyait le mot de passe des comptes de test`
bonus : un build de plus	`expo-updates avant l'unique build`	`un seul build consomme sur le quota gratuit`
eas update --channel preview …	`… --platform android`	`runtimeVersion configuree pour Android seulement`
git log v1.0-web..v2.0	`contient aussi 1 correctif du rapport du Lab 11`	`PR #6, fusionnee apres le tag v1.0-web, qui n'a pas ete deplace`

L'historique pour la soutenance. Le dépôt porte les tags `v1-lab1` `v0.4-spa` `v0.5-supabase` `v0.6-crud` `v0.7-shared` `v0.8-expo` `v0.9-router` `v0.10-mobile-crud` `v1.0-web`. Les Labs 0 à 3 ont été menés avant le dépôt actuel : leurs commits sont rattachés à l'historique (Lab 11), mais seul `v1-lab1` en marque une étape — les tags `v0-init` à `v0.3-effects` du sujet n'existent pas.

13 Ce qui reste à faire par l'équipe

Installer l'APK sur un téléphone Android depuis le lien du README, puis mener les dix points de la recette en 4G, Wi-Fi coupé. Les lignes 4 (session après fermeture) et 10 (deuxième téléphone) ne se vérifient que sur un appareil réel.

Vérifier la mise à jour EAS Update : sur l'écran Profil déconnecté, le sous-titre doit devenir « Le meme compte que sur senevent-mbow.vercel.app. » après deux ouvertures.

L'iPhone reste hors périmètre : installer hors App Store demande un compte Apple Developer payant. Les membres équipés d'un iPhone continuent avec Expo Go — qui demandera, pour le SDK 57, un Xcode ≥ 26.4 sur la machine pour les builds iOS locaux.

Et maintenant : la soutenance

Les deux clients sont en production : le site sur `senevent-mbow.vercel.app`, l'application Android à installer depuis expo.dev. La démonstration croisée — créer un événement sur le web, le voir et s'y inscrire sur le téléphone, et inversement — se fait désormais sans aucun ordinateur de développement. Une logique métier écrite une fois au Lab 7, deux interfaces, une base.